



MoodMatch

feel it. find it.

Project Team

Ismail Yilmaz – 5123090

Phat Vo – 2405645

Erkin Caliskan – 5123096

Md Abdur Rahim – 2407096

May 2026

Contents

1	Introduction	3
2	Core Features	3
3	User Experience & Screen Walkthrough	3
3.1	Welcome Screen	4
3.2	Home Screen	5
3.3	Search Screen	6
3.4	Browse Screen	7
3.5	Detail Screen	8
3.6	Favourites Screen	9
3.7	Settings Screen	10
4	Technical Architecture	11
5	Testing & Continuous Integration	11
5.1	Unit & Widget Testing	11
5.2	Continuous Integration (GitHub Actions)	12

1 Introduction

In today's fast-paced digital landscape, users often face "choice paralysis" when deciding what to watch or listen to. Traditional streaming platforms focus heavily on genres or past viewing history, completely ignoring the most important factor: how the user is feeling right now.

MoodMatch is a cross-platform mobile application built with Flutter that bridges this gap. It acts as a smart, emotion-driven discovery engine. By simply selecting their current mood, users instantly receive a curated trio of recommendations—a movie, a TV series, and a song—tailored to their exact emotional state. This document outlines the application's core features, user experience design, and technical architecture.

2 Core Features

The platform is designed to provide a seamless, offline-first experience with the following key capabilities:

- **Mood-Based Curation:** Users select a mood (e.g., Happy, Calm, Nostalgic) to trigger a dynamically generated mix of content.
- **Multi-Content Discovery:** Movies, TV series, and music are unified into a single, cohesive feed.
- **Smart Search & Filtering:** Real-time input validation and category filters allow users to effortlessly dig into the complete database.
- **Offline-First Architecture:** All recommendation data is bundled locally via JSON, ensuring zero load times and no internet dependency for discovery.
- **Material Design 3 Integration:** A polished, reactive UI that seamlessly adapts to the system's Light and Dark modes.

3 User Experience & Screen Walkthrough

The interface is intentionally minimalist to reduce cognitive load, prioritizing the content and the user's emotional state over cluttered menus.

3.1 Welcome Screen

The entry point of the app sets the tone immediately. It features a full-screen dynamic gradient that responds to the device's theme settings. The MoodMatch logo sits in the center with a subtle drop shadow and hero-animation capabilities. The tagline *"feel it. find it."* sits just above a prominent "Get Started" call-to-action button, ensuring a frictionless onboarding process.

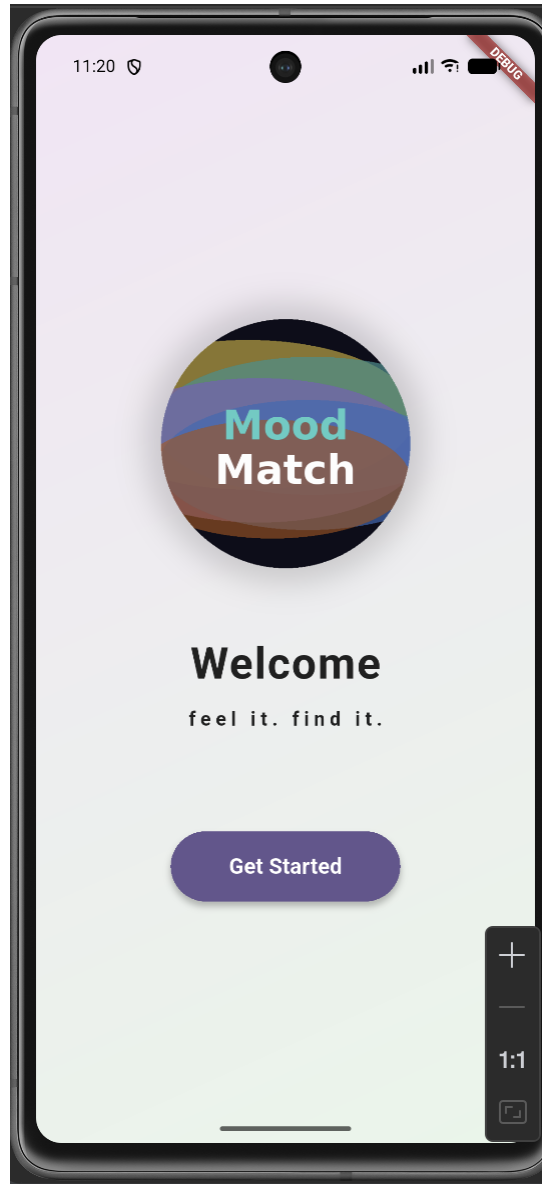


Figure 1: The Welcome Screen with dynamic gradient background.

3.2 Home Screen

The Home Screen is the core engine of the app. It asks a simple question: "How are you feeling?" Users interact with filter chips displaying different moods and emojis. Once selected, a "Mood Trio" appears—a curated top pick for music, movies, and TV series matching that exact feeling. Below this, a "Top Picks" section provides general recommendations for immediate discovery.

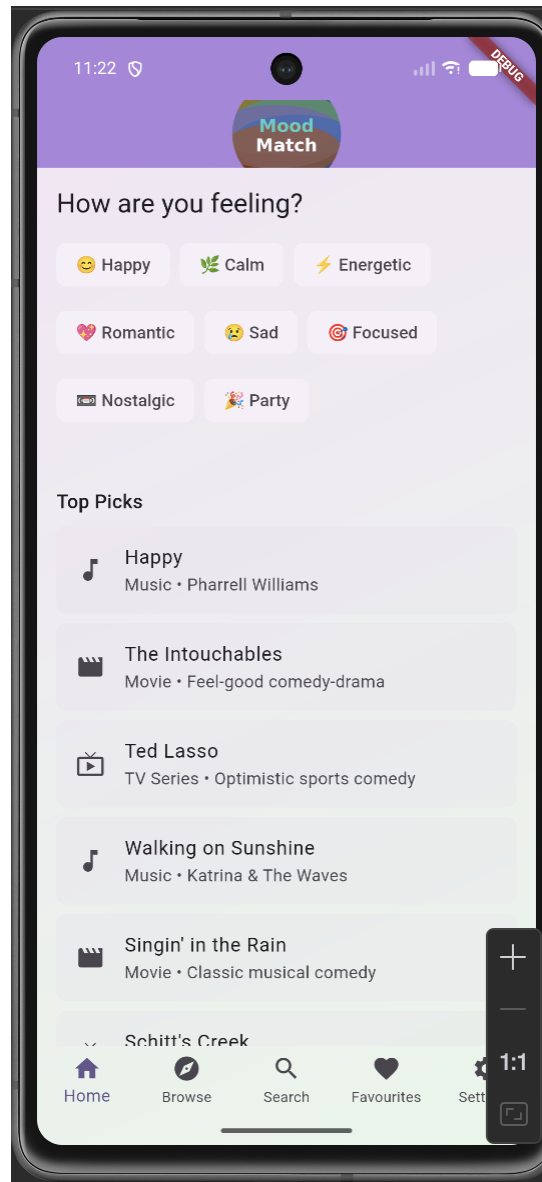


Figure 2: Home Screen showcasing the Mood selection chips.

3.3 Search Screen

The Search Screen provides a robust, real-time discovery tool. It requires a minimum of two characters to initiate a query, ensuring optimized performance. Beneath the search bar, users can apply category filters (Music, Movie, TV Series) that instantly update the results. The screen intelligently manages state, displaying elegant empty states for initial loads, user-friendly messages for "no results found," and clear error handling.

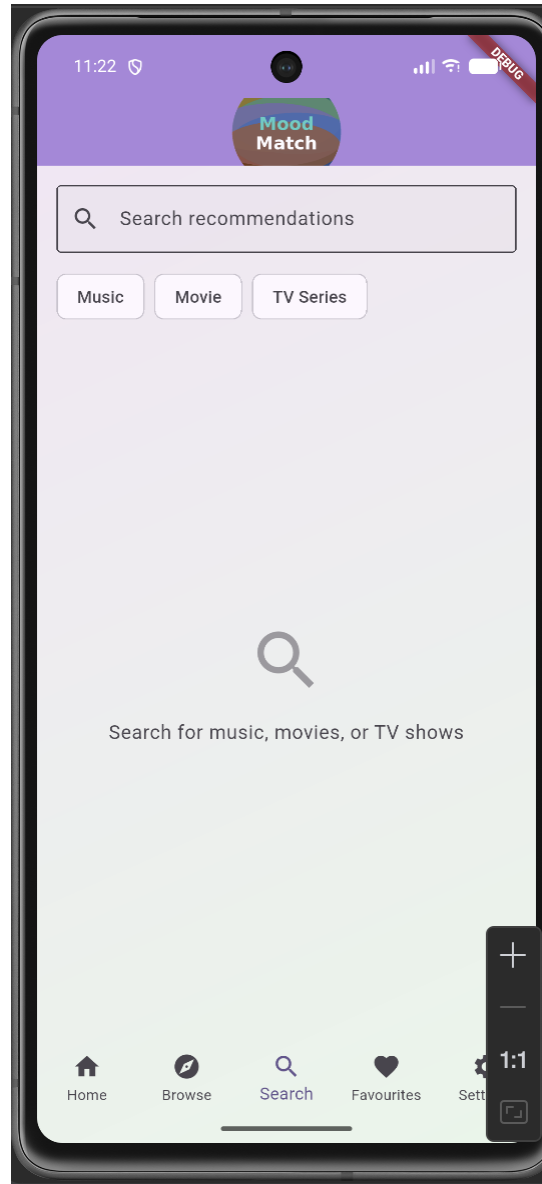


Figure 3: Search Screen featuring real-time validation and dynamic category filters.

3.4 Browse Screen

Built for deep-diving, the Browse Screen lists the entire content library. Horizontal filter chips at the top allow users to isolate specific media types. To ensure optimal performance and user experience, the list uses a `ScrollController` to implement infinite scrolling, automatically fetching the next page of content when the user reaches a 90% scroll threshold.

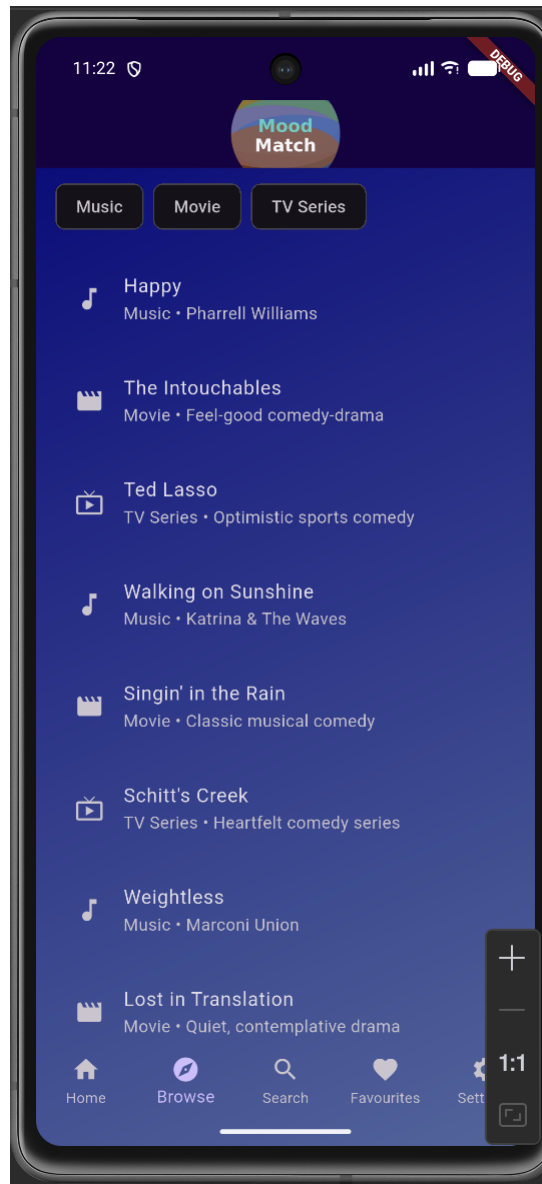


Figure 4: Browse Screen with infinite scrolling and category filters.

3.5 Detail Screen

Tapping any content item pushes the user to the Detail Screen. This view utilizes a **SliverAppBar** that features a large hero image which elegantly collapses into a standard app bar as the user scrolls. It displays the content's title, a dynamically generated rating, the associated mood tag, and a detailed synopsis. A prominent floating action allows users to add the item to their Favourites, complete with a localized **SnackBar** confirmation.

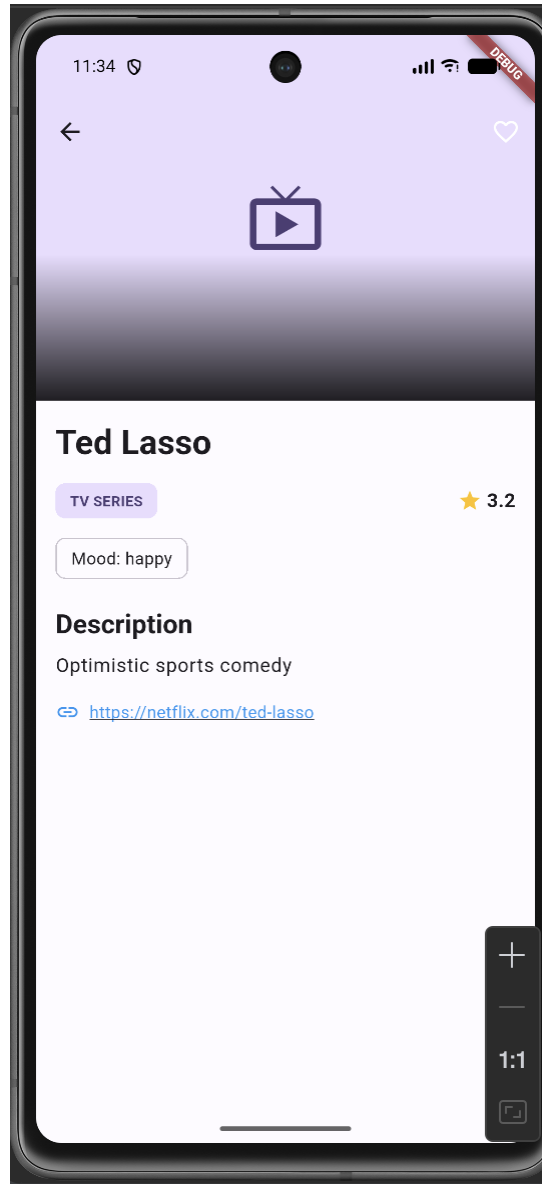


Figure 5: Detail Screen featuring the SliverAppBar and content synopsis.

3.6 Favourites Screen

This screen acts as the user’s personal vault. Powered by local `shared_preferences`, it saves user selections persistently. If the list is empty, a clean, visually appealing ”Empty State” graphic is shown to guide the user. Active list items feature a quick-action trailing icon to remove them from the vault instantly.

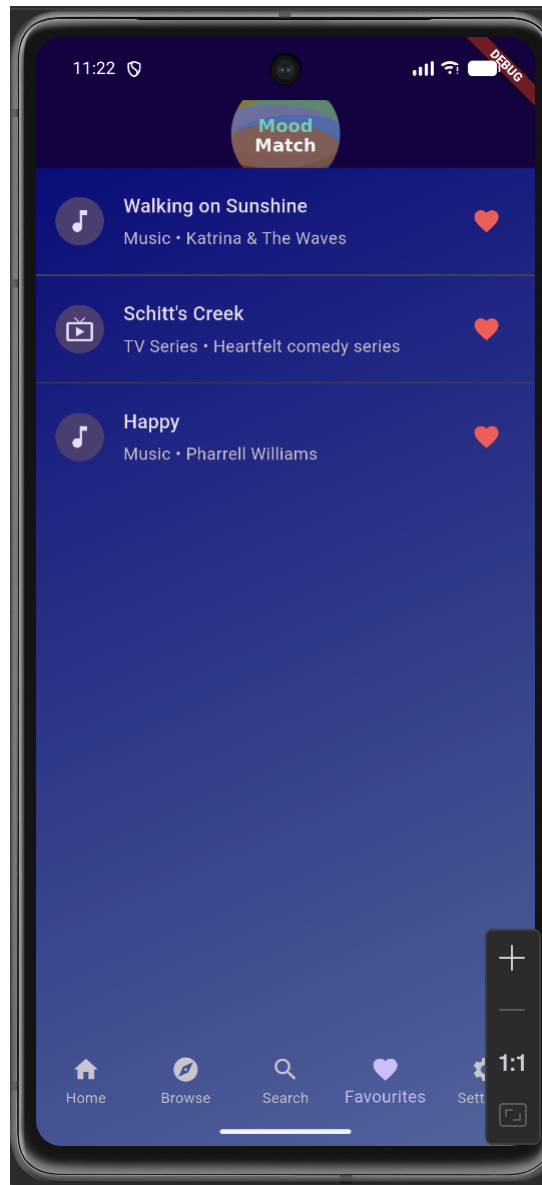


Figure 6: Favourites Screen displaying saved user content.

3.7 Settings Screen

The Settings Screen provides users with direct control over their app experience. It utilizes Riverpod state management to offer an instant, app-wide toggle for Dark Mode, persisting the choice locally. Additionally, it provides clear navigation paths to supplementary information, such as an "About" page containing application credits.

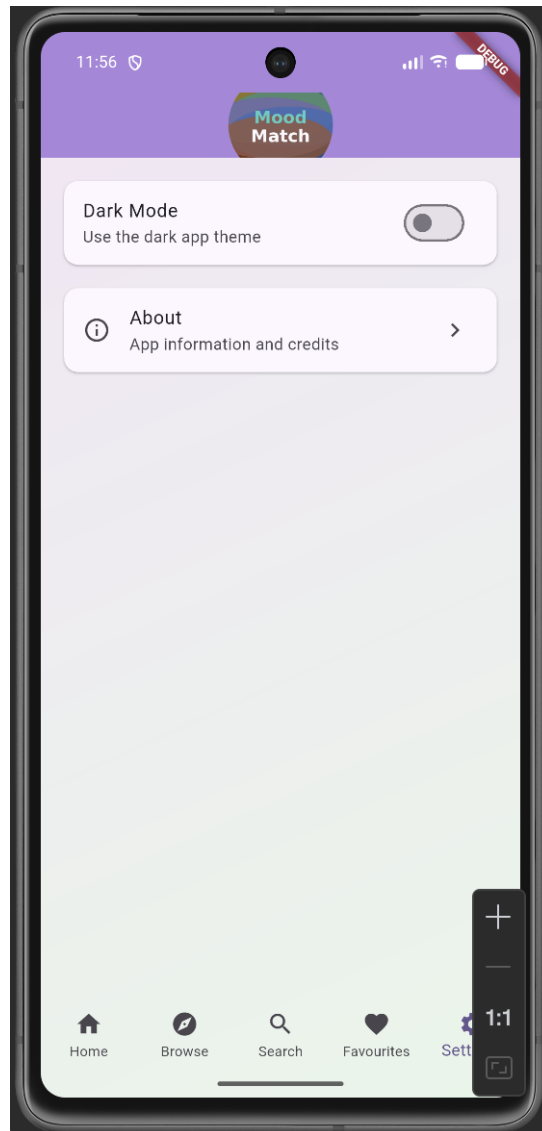


Figure 7: Settings Screen featuring the global Dark Mode toggle.

4 Technical Architecture

To maintain a high standard of code quality and scalability, MoodMatch relies on a strict layered architecture:

- **State Management (Riverpod):** `flutter_riverpod` is used to completely decouple business logic from the UI. Providers like `moodListProvider` and `favoritesProvider` ensure reactive, predictable state updates across the app.
- **Declarative Routing (go_router):** Deep-linking and type-safe navigation are handled via `go_router`, making it highly efficient to pass complex objects (like a `ContentItem`) as extra parameters between screens.
- **Data Handling:** The app follows a strict Repository pattern. The `LocalDataService` parses bundled JSON assets, which are then formatted into strongly-typed Dart models (`Mood`, `ContentItem`) by the `RecommendationRepository`.
- **Centralized Theming (Material 3):** Visual consistency is enforced through a centralized `AppTheme` class. By utilizing `ColorScheme.fromSeed` paired with a primary seed color (`0xFF6750A4`), the app programmatically generates cohesive palettes for both Light and Dark modes. Custom component overlays (e.g., `AppBarTheme`, `CardTheme`, `InputDecorationTheme`) guarantee a pixel-perfect, modern aesthetic across all device settings.

5 Testing & Continuous Integration

To ensure the reliability and maintainability of the application, MoodMatch incorporates a robust testing strategy alongside an automated Continuous Integration (CI) pipeline.

5.1 Unit & Widget Testing

The testing suite is divided into specific domains to validate both business logic and user interface components:

- **Model Tests (`content_item_test.dart`):** Validates the serialization and deserialization (`fromJson` / `toJson`) of core data models, ensuring that JSON assets are correctly mapped to strongly-typed Dart objects.
- **Provider Tests (`theme_provider_test.dart`):** Tests the Riverpod state notifiers. This ensures that features like dynamic theme toggling (Light/Dark mode) accurately update the state and persist user preferences correctly via `SharedPreferences`.
- **Widget Tests (`detail_screen_test.dart`):** Simulates user interactions in a headless environment. These tests verify that the `DetailScreen` renders the correct titles, subtitles, and icons, and validates that tapping the favorite button successfully updates the UI state and triggers the `SnackBar` confirmation.

5.2 Continuous Integration (GitHub Actions)

A GitHub Actions workflow (`flutter_ci.yml`) serves as a gatekeeper for the codebase. Triggered automatically on every push and pull request to the `main` or `master` branches, the pipeline runs the following checks on an Ubuntu runner:

1. **Dependency Resolution:** Fetches all packages via `flutter pub get`.
2. **Static Analysis:** Executes `flutter analyze` to enforce clean architecture, catch syntax errors, and maintain readability standards.
3. **Test Execution:** Runs `flutter test` to guarantee that no breaking changes are merged into production. If any unit or widget test fails, the CI build halts, protecting the integrity of the application.